

Python Programming with Artificial Intelligence

AI Coding Agent

Huangxun Chen @IoT&AI Thrust



- In last lecture, we have learned several practical techniques and frameworks for building LLM-powered applications and integrating LLM capabilities into Python programs.
- **AI coding agents** are specialized LLM-powered applications that can understand software development tasks and assist with coding, debugging, testing, and code maintenance.
- In today's lecture, we will learn how to use AI coding agents.

Workflow Type

- Core Agent Loop: read, reason, act, evaluate
- Coding Agent Workflows: IDE Agent, Terminal Agents, Pull Request Agents, Cloud Agents

Terminal Agent (Using GitHub Copilot CLI)

- Install & Authenticate With GitHub
- Slash Commands & Keyboard Shortcut
- Agent Modes (Standard & Plan & Autopilot), Sub-Agent
- Switch Between AI Providers

Pull Request Agents (Using GitHub Copilot code review)

- Request a Code Review (--reviewer @copilot), Review and Apply Copilot's Suggestions
- Configure Copilot for Project (repository settings & custom instructions)

Context Engineering for Coding Agent

- Context-Management Strategies: Curate, Distill, Delegate, Externalize



Workflow Type

■ ■ ■ Understanding AI Coding Agents

Standard chatbots provide one-off answers, coding agents are designed for autonomy, operating through a continuous execution loop to solve complex coding tasks.

This loop typically follows **four distinct steps**:

- **Read**: They read relevant files from your codebase to form their context.
- **Reason**: They determine the logical steps needed to achieve your goal.
- **Act**: They execute those steps by editing files, running terminal commands, or using external tools.
- **Evaluate**: They check the results of their actions to see if more work is needed.

This loop repeats until the task is completed or the agent hands control back to you.

Where an Agent runs

The core agent loop will generally stay the same, but where an agent runs will shape how you interact with it.

In an editor: it works alongside you.

→ **IDE Agent**

In a terminal, you guide it step by step.

→ **Terminal Agents**

In pull requests, it reviews changes asynchronously.

→ **PR Agents**

In the cloud, it works in a managed environment and reports back. → **Cloud Agents**

- IDE agents live **inside your code editor** and work alongside you in real time.
- They suggest edits inline, show visual diffs, and let you accept or reject changes without leaving your editing environment.

- IDE agents live **inside your code editor** and work alongside you in real time.
- They suggest edits inline, show visual diffs, and let you accept or reject changes without leaving your editing environment.

Two Types:

- **AI-native IDE:** built from the ground up around AI capabilities
 - Typical Products: Cursor, Windsurf, ...
 - **IDE Integration:** add agent features to editors you already use.
 - Typical Products: Github Copilot extension, Gemini Code Assist, Claude Code in VS Code/JetBrains extension
- * cloud-backed IDE agents send your code to external servers for processing.

- Terminal agents run **in your shell**.
- Terminal agents works well for complex changes and navigating large codebases.
 - You describe a task, and the agent reads files, proposes edits, and runs commands.
 - You generally approve or reject each step before the agent moves on.
- Typical Products: GitHub Copilot CLI, Claude Code, Gemini CLI, Codex CLI, Cursor CLI, OpenCode

Pull Request Agents

- PR-agent typically work **on version control platforms** like GitHub, GitLab,
- PR-agent generally operates on shared branches visible to everyone on the team, not on your local workspace.
- The agent is asynchronous, often triggers automatically when a pull request is opened or updated.
- The agent flags potential bugs, style violations, and logic issues, but a human reviewer makes the final call on whether to merge.
- Typical Products: GitHub Copilot code review, CodeRabbit

- Cloud agent works **in a remote or managed environment.**
- Cloud agents are used for greenfield prototyping or work that takes longer than you would want to sit and supervise.
 - You describe a task, the agent works in the cloud, and later reports back with a branch, pull request, or prototype.
 - You can often access cloud agents through Slack, issue trackers, or a web browser.
- Many cloud agents rely on vendor-managed infrastructure, other cloud agents can run against machines you control.
- Typical Products: GitHub's Copilot cloud agent, Claude Code on the web, Codex web, Cursor's Cloud Agents.

Coding Agent Workflows Summary

- Workflow types describe interaction modes
- They don't always map cleanly to product categories.

	Where it runs	Interaction Mode	Autonomy Level	Best For
IDE Agent	Editor	Interactive	Supervised	Quick Edits
Terminal Agent	Terminal	Conversational	Supervised	Multi-file changes
PR Agent	GitHub/GitLab	Async	Semi-Autonomous	Code review
Cloud Agent	Cloud	Fire-and-forget	Highly-Autonomous	Long-running Task

- Assuming one agent type handles everything
 - Matching the workflow to the task matters more than picking a single tool.
- Ignoring privacy and compliance constraints
 - Many cloud agents run on remote infrastructure.
 - PR agents operate in shared repositories.
 - IDE and terminal agents with cloud backends send code to external APIs for inference.
 - Some terminal agents can connect to local models through tools like Ollama.
- Over-automating without review
 - Always assume AI-generated code contains mistakes.
 - Review all generated code carefully before merging.



Terminal Agent (Using GitHub Copilot CLI)

- Copilot CLI is an AI-powered agent for coding workflows in the terminal.
- It answers questions, writes and debugs code, and interacts with GitHub.com services through natural-language prompts.

You need:

- A GitHub account
- An active Copilot subscription
- Basic terminal familiarity: running commands in a terminal.
- Download the sample project from the Canvas.

Install Copilot CLI

- The most portable way to install Copilot CLI is from a terminal.
- During the installation, the script may ask if you want to add Copilot CLI to your PATH. If you say yes, then it adds the install location to your shell profile so you can run copilot from any terminal window.

macOS + linux

```
$ curl -fsSL https://gh.io/copilot-install | bash  
$ # Or with wget if curl isn't available:  
$ wget -qO- https://gh.io/copilot-install | bash
```

Windows

```
[Windows PowerShell]  
PS> winget install GitHub.Copilot
```

After install, close and reopen your terminal, verify:

```
$ copilot --version
```

Authenticate With GitHub

- Most of your work with Copilot CLI happens inside an interactive session where you have a conversation with the agent.
- To get started, run copilot with no arguments from the directory you want to work in:

```
$ copilot
```

- Then, you first need to sign in with the /login command:

```
> /login
```

- Choose GitHub.com and press Enter.
- Copilot will generate a code and provide a login URL.
- Open the URL in your browser, and complete the GitHub OAuth flow.

Authenticate With GitHub

- To confirm everything is wired up, ask Copilot a quick question:

› What version of Copilot CLI are you?

Fetching the official Copilot CLI documentation to confirm the agent version and provide an authoritative answer. I'll call the docs tool now.

Checking my documentation # GitHub Copilot CLI Documentation

According to the Copilot CLI documentation, the CLI version in this environment is 1.0.71.
I'm powered by gpt-5-mini (model ID: gpt-5-mini).

- You can run the `/exit` command to leave Copilot CLI and return to your shell.

› `/exit`

Alternative Mode: Use One-Shot Prompts

- For quick questions or scriptable tasks, pass your prompt directly with the **-p option**
- This one-shot style is great for quick lookups or when you want to integrate Copilot into a shell script.
- The output includes the agent's response, a summary of any file changes, stats about the requests, and a token usage breakdown.

```
$ copilot -p "Define Python in one sentence."
```

```
Python is a high-level, interpreted programming language known for its simple, readable syntax that emphasizes code clarity and allows developers to write programs using fewer lines of code compared to languages like Java or C++.
```

```
Changes   +0 -0
```

```
AI Credits 2.76 (8s)
```

```
Tokens    ↑ 21.5k (21.5k written) • ↓ 131 (83 reasoning)
```

Useful Slash Commands

- Run copilot to start a new session.
- Inside the UI, you will have access to slash commands that allow you to control the session itself and your current project.

Command	Description
/init	Initialize custom instructions and agentic features
/new	Clear the conversation history and start a new session
/model	Select or switch the AI model mid-session
/diff	Review the changes made in the current directory
/review	Run the code-review agent to analyze your changes
/agent	Browse and select from available agents
/exit	Exit the CLI session
/logout	Log out of Copilot
/help	Show help for all available interactive commands

Useful Keyboard Shortcuts

- Run copilot to start a new session.
- Inside the UI, you will have access to slash commands that allow you to control the session itself and your current project.

Shortcut	Action
/	Access slash commands
@	Mention files
#	Mention issues and pull requests
!	Execute a shell command
↑ Shift + Tab ↹	Switch between agent modes
Esc	Cancel the current operation
^Ctrl + L	Clear the screen
^Ctrl + T	Toggle reasoning display

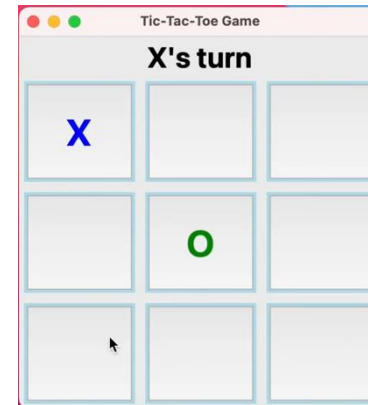


Use Agents to Explore and Modify
Codebase

■ Use Agents to Explore and Modify Codebase

- Decompress the the sample Python project into a tic-tac-toe/ directory.
- The archive is actually a Git repository with two branches:
 - Main: holds the original code,
 - copilot-cli: contains all the changes made with Copilot CLI for comparison.
- With the project folder in place, navigate into it and start a Copilot CLI session:

```
$ cd tic-tac-toe/  
$ copilot
```



Switch Between AI Providers

- Copilot CLI gives you access to models from OpenAI and Anthropic.
- To switch to a different model, run `/model` inside an interactive session and pick from the list:

```
> /model
```

The initial picker lets you select the model.

The second picker allows you to choose the reasoning effort.

- The reasoning effort controls how long the model thinks before it responds.
- More reasoning also consumes more tokens.

If you'd rather not pick manually, then select the **Auto** option.

- Copilot choose the best model for each request based on the task and current system load

■ Explore Your Codebase With Copilot CLI

Ask Copilot to give you a summary of the codebase.

➤ Explore this project and tell me what it does.

This is a simple GUI Tic-Tac-Toe implemented in one Python script (main.py) using Tkinter.

- TicTacToeGame: game logic (3x3 board, move validation, win/tie detection, reset, player toggle).
- TicTacToeBoard: Tkinter GUI (menu: Play Again/Exit, status label, 3 × 3 button grid, click handling, highlight winning combo).
- Players are X (blue) and O (green). Run with: `python main.py` (requires Tkinter, part of standard Python on most platforms).

Initialize the Agent Instructions

- For the first time you are working on a project, you should create customized agent instructions based on the project.
- You can do this using the `/init` command.

```
> /init
```

- Once you press Enter, the agent will scan your project and create the instruction file. By default, this file will live at `.github/copilot-instructions.md`.
- You can tweak the file as you wish.
- Whenever you start a new Copilot session in the project directory, the agent will automatically load this file and add it to its context for working on the project.

```
$ git add .github/copilot-instructions.md  
$ git commit -m "Initialize the agent instructions"
```

Understand Subagents

Behind the main agent, Copilot CLI also has a roster of built-in subagents: internal workers that the main agent calls on for focused tasks.

- You won't invoke these subagents directly most of the time.
- Copilot picks the right subagent for each request.

Sub Agent	Description
explore	Performs quick codebase analysis so you can ask questions about your code without adding to the main context.
task	Runs commands such as tests and builds, providing a brief summary on success and the full output on failure.
general-purpose	Mirrors the main agent's full capabilities to handle complex, multi-step tasks in a separate context, keeping your main conversation focused.
code-review	Reviews changes with a focus on surfacing only genuine issues, minimizing noise.
research	Performs deep research across your codebase, related repositories, and the web. It's invoked when you explicitly call the /research slash command.
rubber-duck	Acts as a constructive critic powered by a different model family.

Code Review

Depending on the project size and your prompt, Copilot may delegate this task to the code-review subagent or handle it inside the main agent.

➤ Review the code for bugs, edge cases, and anything that could be simplified. Focus on real issues, not style.

In-Class Practice: Check the Review Results

Analyze each review comment with a critical eye and make your own call.

For example, you can pick one suggestion and ask Copilot to fix it.

› Let's fix the xxx issue

- Copilot will propose a change and ask for your approval before writing to the file.
- If you accept, then it makes the change.
- When the change is done, the agent may run a quick test to verify that the issue is resolved.
- To see all the changes made so far, run the `/diff` command
- To leave Diff Mode, press Esc.

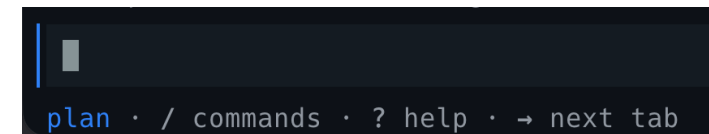
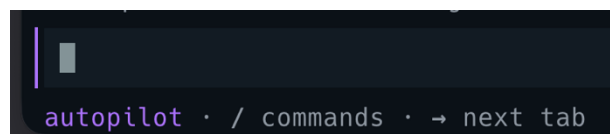
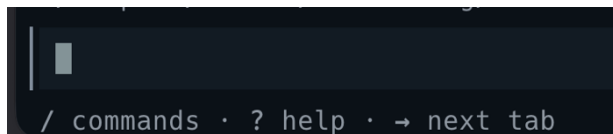
› `/diff`

■ Understand Agent Modes

Modes that control how the main agent behaves.

Copilot CLI has three modes:

- **Standard (Default)**: The default ask-and-execute mode for general conversation and direct code generation.
 - **Plan**: A read-only mode that confines the agent to producing and refining an implementation plan, with no file edits, command runs, or other side effects.
 - **Autopilot**: A mode that allows Copilot to work autonomously, performing multiple steps to complete a goal on your behalf.
-
- The current mode is shown in the status bar at the bottom of the screen.
 - You **switch modes by pressing Shift+Tab**.



Plan Before You Act

- For big refactors or new features, it's worth taking some time to build a coherent action plan.
- You can use the agent for this planning stage.
- Press Shift+Tab once to **switch to Plan mode** and then describe what you want to achieve.

➤ Add a score tracker that counts wins for X and O across multiple rounds.

Plan Before You Act

- In **Plan** mode, Copilot does not touch your files.
- Instead, it analyzes the request, sometimes asks clarifying questions, and produces a structured plan you can review.
- If the plan looks off, you can refine it in plain English until it matches your intent.
- Once you've refined it, press Shift+Tab to switch to **Standard** or **Autopilot** mode and then ask Copilot to execute the plan.

Autopilot Mode

In autopilot mode, the agent works through the plan with minimal interruption, checking in less often than in the other modes.

- You must review the final diff carefully before committing.
- If something goes wrong, then double-tap Esc to rewind the session to a previous state and try a different approach.

Autopilot Mode

In Autopilot, if a step needs a permission you haven't granted, like writing files, running shell commands, or reaching a URL, agent denies it automatically instead of pausing to ask. That can stall the task halfway through.

To pre-approve everything for the session, run the **/yolo** command:

```
/yolo
```

- The **/yolo** command is a **session-wide permission grant** equivalent to **/allow-all**.
- Once it's on, the agent can use any tool, touch any file, and reach any URL without asking.
- The grant **survives /new** and **/restart**, so you only need to enable it once per session.
- To check the current state, run **/yolo show**. To revoke it, run **/yolo off**.

Run a Fleet of Agents

- Copilot CLI can also run a task across multiple agents with the `/fleet` command.
- A natural way to use `/fleet` is to run it after you build a plan using Plan mode.
- If the finished plan has multiple independent elements, then Copilot will offer an Accept plan and build on autopilot + `/fleet` option.

Practical Tips

- Before you switch to a new task, it's best practice to clear your conversation with the `/new` or `/clear` command.
- A fresh context keeps the agent focused and avoids confusion from earlier, irrelevant exchanges.



Pull Request Agents
(Using GitHub Copilot code review)

Pull Requests

- Pull requests are the standard collaborative workflow provided by GitHub and similar services like GitLab to facilitate code review for projects managed with Git.
- A pull request, or a PR for short, is a formal request to merge code from one branch, or fork, into another, and it's where code review typically happens.

■ GitHub Copilot code review TBD

- Rather than waiting for a teammate, you can add Copilot as a reviewer to provide an AI-powered code review shortly after you open a pull request on GitHub.
- Downloading the sample project from canvas
- Upload the local pop-quiz repository, including all branches, to your GitHub account
- There is a feature branch with intentional code issues that GitHub Copilot can catch when you request a review.

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"  
$ brew install gh  
$ gh auth logi  
  
$ gh repo create pop-quiz --source=. --remote=origin --public --push  
$ git push --all origin
```

GitHub Copilot: Request a Code Review

- To see GitHub Copilot code review in action, you'll open a pull request from a feature branch into the main branch and assign Copilot as the reviewer.

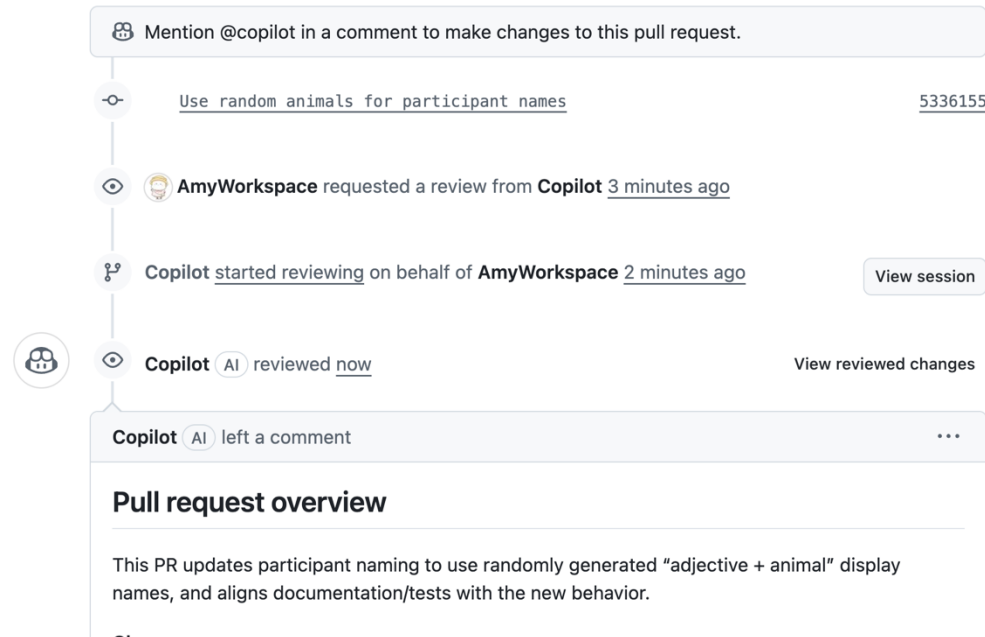
```
$ git switch feature/player-names  
Switched to branch 'feature/player-names'  
Your branch is up to date with 'origin/feature/player-names'.
```

```
$ gh pr create --fill --reviewer @copilot
```

```
Creating pull request for feature/player-names into main in AmyWorkspace/aiaa3102-pop-quiz  
https://github.com/AmyWorkspace/aiaa3102-pop-quiz/pull/1
```

GitHub Copilot: Request a Code Review

- Once the command succeeds, you'll be presented with a link to your pull request in GitHub.
- Copilot will get to work automatically. You can track its progress in the GitHub Actions section of your repository.
- Copilot only leaves comments, never approving or requesting changes.
- It's worth noting that Copilot reviews consume your monthly token quota.



GitHub Copilot: Review and Apply Copilot's Suggestions

- GitHub adds Copilot's feedback to the Conversation tab of your pull request.
- You'll see a top-level summary of the changes in your PR written by the agent, followed by inline comments anchored to specific lines in the diff.
- Like all AI agents, Copilot is non-deterministic in its reviews.

Copilot AI left a comment

Pull request overview

This PR updates participant naming to use randomly generated "adjective + animal" display names, and aligns documentation/tests with the new behavior.

Changes:

- Add `pop_quiz.animals.random_animal_name()` and tests for it.
- Update `GameState.add_participant()` to assign animal-based participant display names.
- Update README to describe the new naming behavior.

Reviewed changes

Copilot reviewed 5 out of 5 changed files in this pull request and generated 3 comments.

► Show a summary per file

💡 Add Copilot custom instructions for smarter, more guided reviews. [Learn how to get started.](#)

🔗 Fix batch with Copilot

src/pop_quiz/animals.py

Comment on lines +65 to +67

```
65 + adjective = random.choice(_ADJECTIVES)
66 + index = random.randint(1, len(_ANIMALS) - 1)
67 + return f"{adjective} {_ANIMALS[index]}"
```

Copilot AI 1m ago

Low

`random_animal_name()` never returns the first entry in `_ANIMALS` because it uses `randint(1, len(_ANIMALS) - 1)`, making index 0 unreachable. This biases the distribution and makes the first animal effectively dead code.

Suggested changeset 1

src/pop_quiz/animals.py

...	@@ -63,5 +63,5 @@
63	63 def random_animal_name() -> str:
64	64 """Return a random animal name like 'Brave 🐯 Tiger'."""
65	65 adjective = random.choice(_ADJECTIVES)
66	- index = random.randint(1, len(_ANIMALS) - 1)
67	- return f"{adjective} {_ANIMALS[index]}"
66	+ animal = random.choice(_ANIMALS)
67	+ return f"{adjective} {animal}"

Commit suggestion Add suggestion to batch

   Copilot uses AI. Check for mistakes.

GitHub Copilot: Review and Apply Copilot's Suggestions

When a Copilot comment includes a suggested change, you'll see a **Commit suggestion** button right under the diff.

- Clicking it stages the change and lets you write a commit message before pushing.
- Once you commit, GitHub pushes a new commit to your feature/player-names branch on your behalf.
- The pull request updates automatically, the resolved conversation collapses, and Copilot's earlier comment is marked Outdated.

src/pop_quiz/animals.py

Comment on lines +65 to +67

```
65 + adjective = random.choice(_ADJECTIVES)
66 + index = random.randint(1, len(_ANIMALS) - 1)
67 + return f"{adjective} {_ANIMALS[index]}"
```

Copilot AI 1m ago Low ...

random_animal_name() never returns the first entry in _ANIMALS because it uses randint(1, len(_ANIMALS) - 1), making index 0 unreachable. This biases the distribution and makes the first animal effectively dead code.

Suggested changeset 1

src/pop_quiz/animals.py

```
@@ -63,5 +63,5 @@
63 63 def random_animal_name() -> str:
64 64     """Return a random animal name like 'Brave 🐅 Tiger'."""
65 65     adjective = random.choice(_ADJECTIVES)
66 - index = random.randint(1, len(_ANIMALS) - 1)
67 - return f"{adjective} {_ANIMALS[index]}"
66 + animal = random.choice(_ANIMALS)
67 + return f"{adjective} {animal}"
```

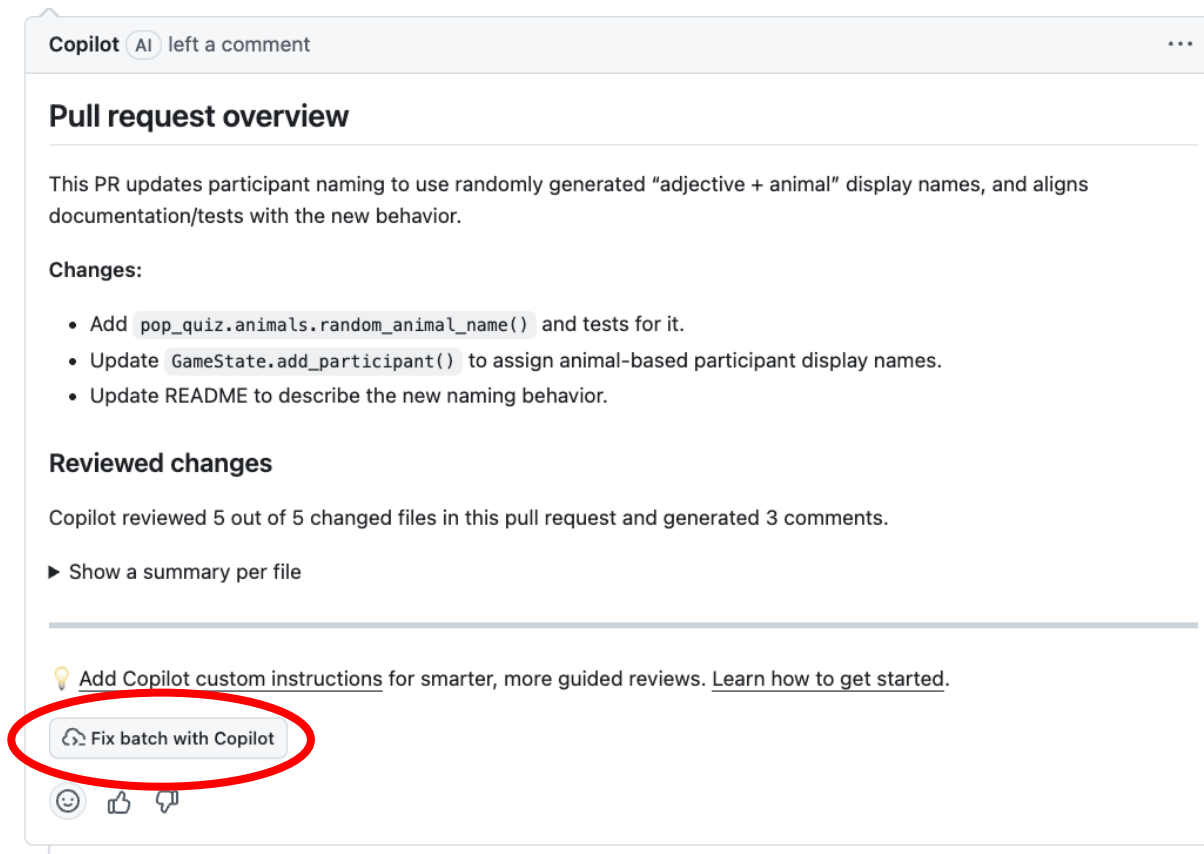
Commit suggestion Add suggestion to batch

Copilot uses AI. Check for mistakes.

GitHub Copilot: Review and Apply Copilot's Suggestions

If you don't want to make the changes yourself, then you can hand the review off to the Copilot coding agent.

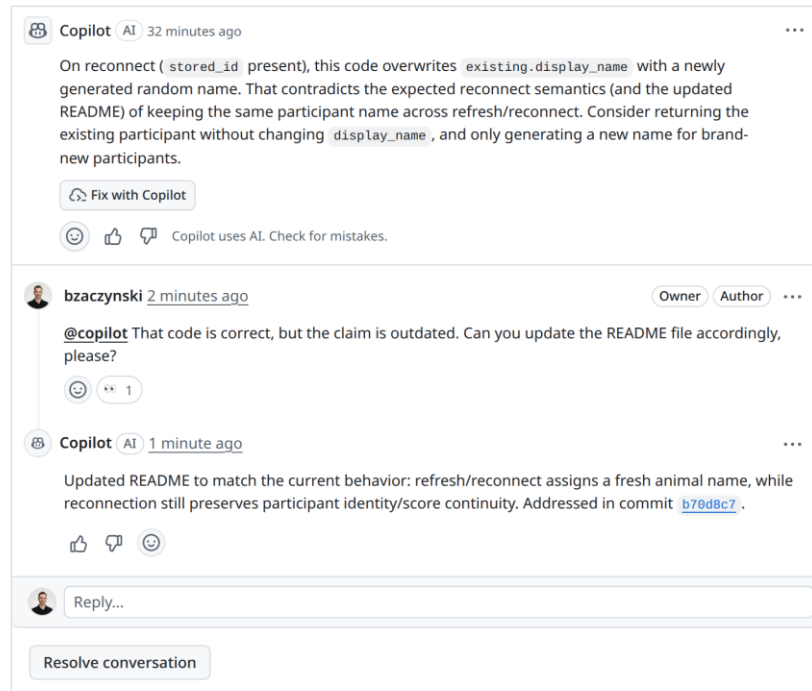
- Return to the Conversation tab, find the summary comment at the beginning of the thread, and click the **Fix batch with Copilot** button to let the agent do its work in the cloud.



GitHub Copilot: Review and Apply Copilot's Suggestions

Sometimes, Copilot's comment isn't quite right.

- You can reply to any Copilot comment by mentioning @copilot directly in the thread.
- The agent will acknowledge your comment by reacting to it with the eyes emoji and immediately get to work.
- After a minute or two, Copilot will post a follow-up reply, just like a human reviewer.
- If a suggestion doesn't make sense, click Resolve conversation without applying it, optionally adding a short reply explaining why.
- The conversation collapses, and the thread no longer blocks your attention.



Configure Copilot for Project

Requirement:

- let Copilot to run automatically on every pull request
- Follow our conventions instead of generic best practices.

Solution: GitHub supports both through **repository settings** and **custom instructions** in the special `.github/` folder inside the project.

Repository Settings

- Open the pop-quiz repository on GitHub and click the **Settings** tab.
- Next, in the left sidebar, under **Code, planning, automation**, expand **Rules** and click **Rulesets**.
- Click **New ruleset** and choose **New branch ruleset**.
- Give the ruleset a name, such as “Copilot Auto Review,” and set **Enforcement Status to Active**.
- Under Target branches, add the branches you want to cover, like the default branch.
- Scroll down to the bottom of Branch rules and select the **Automatically request Copilot code review** option.
- Then, configure the optional sub-settings and click Create.

☒ **Automatically request Copilot code review**
Request Copilot code review for new pull requests automatically if the author has access to Copilot code review and their premium requests quota has not reached the limit.

Hide additional settings ^

☒ **Review new pushes**
Copilot automatically reviews each new push to the pull request.

☒ **Review draft pull requests**
Copilot automatically reviews draft pull requests before they are marked as ready for review.

Repository Settings

- Open the pop-quiz repository on GitHub and click the **Settings** tab.
- Next, in the left sidebar, under **Code, planning, automation**, expand **Rules** and click **Rulesets**.
- Click **New ruleset** and choose **New branch ruleset**.
- Give the ruleset a name, such as “Copilot Auto Review,” and set **Enforcement Status to Active**.
- Under Target branches, add the branches you want to cover, like the default branch.
- Scroll down to the bottom of Branch rules and select the **Automatically request Copilot code review** option.
- Then, configure the optional sub-settings and click Create.

In-Class Practice: create a new PR without explicitly specifying Copilot as the reviewer.

Custom Instructions

- Out of the box, Copilot reviews using a generic notion of good code.
- You can sharpen its feedback by writing a `.github/copilot-instructions.md` file at the root of your repository.
- The agent reads this file before every review and treats its contents as project-wide guidance.
- Each instruction is a hint, not a hard rule. Copilot will weigh your instructions against the diff and surface comments where the gap is largest.

Pop Quiz – Copilot Review Instructions

This project is maintained by a Polish-speaking team.

To keep code reviews accessible to everyone, please follow the guidelines below.

Language

- Write ****all review comments in Polish****, including the summary comment.
- Keep code identifiers, log messages, and docstrings in English.
- Use the informal register rather than the formal one.

Custom Instructions

- Out of the box, Copilot reviews using a generic notion of good code.
- You can sharpen its feedback by writing a `.github/copilot-instructions.md` file at the root of your repository.
- The agent reads this file before every review and treats its contents as project-wide guidance.
- Each instruction is a hint, not a hard rule. Copilot will weigh your instructions against the diff and surface comments where the gap is largest.

Pop Quiz – Copilot Review Instructions

This project is maintained by a Polish-speaking team.

To keep code reviews accessible to everyone, please follow the guidelines below.

Language

- Write ****all review comments in Polish****, including the summary comment.
- Keep code identifiers, log messages, and docstrings in English.
- Use the informal register rather than the formal one.

In-Class Practice: create a new PR with new instructions.md, and check the copilot response.

Custom Instructions

- For larger codebases, GitHub supports path-specific instructions via files under `.github/instructions/`, each named with **an `.instructions.md` suffix and a glob in the front matter**.
- Copilot loads the matching files for any pull request that touches paths covered by their `applyTo` glob.

```
[instructions/frontend.instructions.md]
```

```
---  
applyTo: "src/pop_quiz/static/*.js"  
---
```

The frontend supports browsers from the last two major versions of Safari, Chrome, and Firefox. Avoid suggestions that depend on features without broad support, such as top-level `await` or class private methods.

Copilot CLI: Work Directly With GitHub Issues and Pull Requests

- CLI authenticates against your GitHub account, it has native access to your repositories, issues, and pull requests.
- You can ask Copilot CLI to fix an issue by its number:

› Fix issue #42

- Copilot reads the issue, reviews the relevant files, and edits them in your working directory.
- When you're ready to share the work, the `/pr create` command can open a pull request.

Practical Tips

- **Keep PRs small and focused:** Copilot's reviews work best on small pull requests when the diff fits the model's effective context window. If a PR touches more than a few hundred lines or spans many unrelated files, split it into smaller pieces. As a side benefit, smaller PRs are easier for human reviewers, too.
- **Re-request the review after pushing changes:** When you address Copilot's comments and push new commits, the agent doesn't automatically re-run on the updated diff. Click Re-request review next to Copilot's name in the Reviewers sidebar to ask for a fresh pass.
- **Keep humans in the loop:** Copilot comments only, never approving or requesting changes. Treat it as an extra pair of eyes on top of your usual human review process.
- **Watch your token usage**



Context Engineering for Coding Agent

Modern AI coding agents work with a **fixed-size context window**.

Context engineering is the practice of curating the content that goes into an AI agent's context window.

Compared with Prompt Engineering

- Prompt engineering is about crafting individual prompts.
- Context engineering manages everything in the context window.

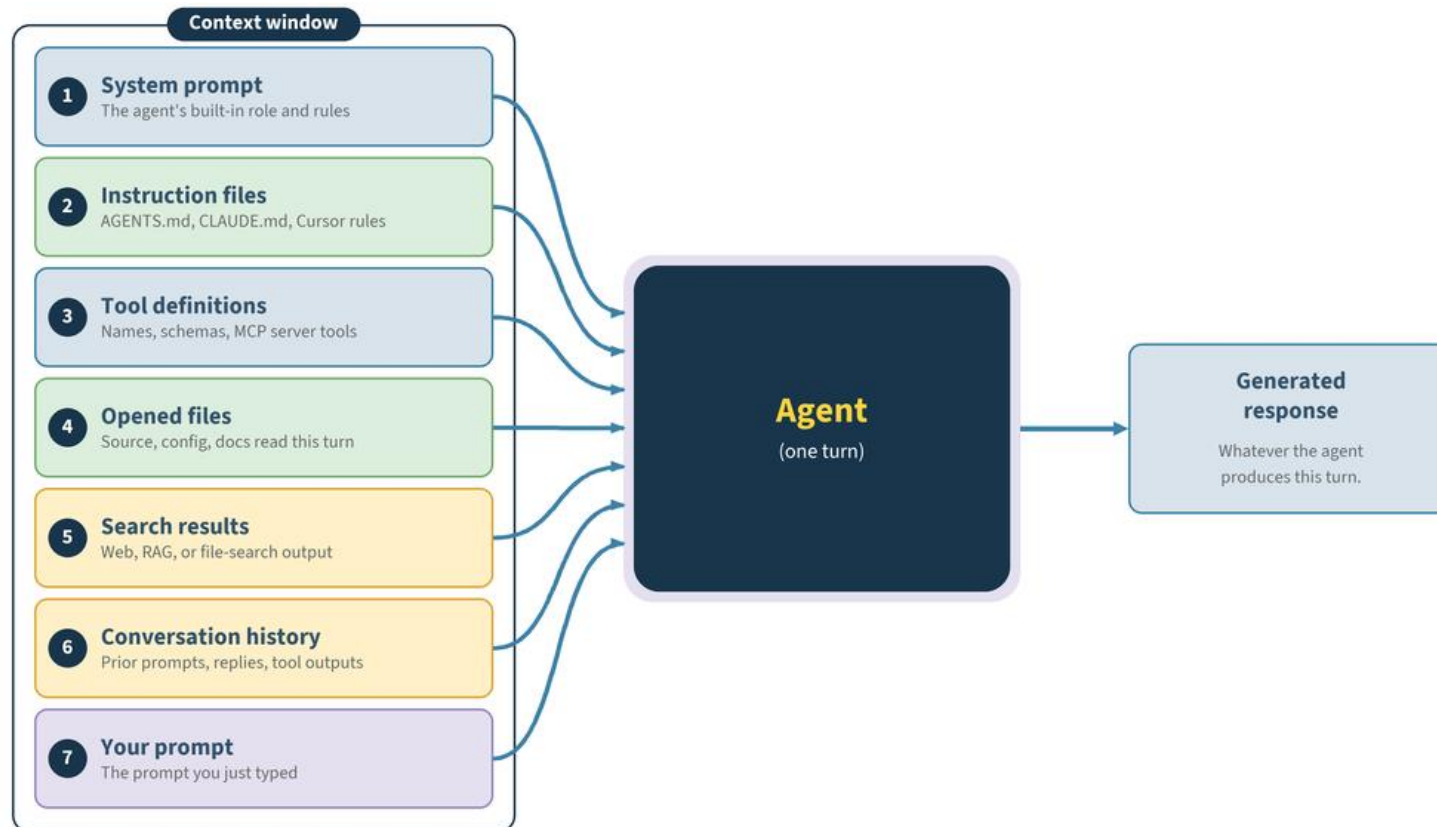
The context window is **everything the agent sees on a single turn**.

On any given turn, whenever you write a prompt and press Enter, the agent's window typically holds the following layers of content:

- **System prompt:** The agent's built-in role and rules, typically set by the vendor.
- **Instruction files:** Files auto-loaded into every turn.
 - e.g. CLAUDE.md, Cursor rules, Copilot instructions, Skill descriptions
- **Tool definitions:** Names, short descriptions, and JSON Schema definitions for every tool the agent can call, and, in some frameworks, any subagent the main agent can delegate to.
- **Opened files:** Source files, configuration files, or docs the agent has read in this session.
- **Search results:** Results from web searches, internal RAG queries, or file searches.
- **Conversation history:** Earlier prompts, replies, and tool outputs of the same session.
- **Your new prompt:** The prompt you just typed, plus any attachments.

Context for AI Coding Agents

The context window is **everything the agent sees on a single turn**.
On any given turn, whenever you write a prompt and press Enter, the agent's window typically holds the following layers of content:



Properties of Context Window

Fixed: The context window can **only hold a certain number of tokens**. Everything above that number competes for the same budget.

Position-sensitive: **Items at the top and bottom of the window tend to get more attention** from the model than items in the middle. Long conversation histories may push important rules into the dead zone in the middle of the context window. *

Non-persistent: Most of the context evaporates when you close the session. What survives is whatever you write to disk or a memory store.

* [TACL 2023] Lost in the Middle: How Language Models Use Long Contexts

Common Context-Engineering Failures

The agent forgot your rules

- Older context fades as the window fills, and your earliest instructions get crowded out.

The agent picks the wrong tool.

- Too many similar options crowd out the right one.

The agent gets conflicting signals.

- Two pieces of context disagree, and the agent picks one randomly.

The agent quotes its own hallucination

- A wrong answer becomes fact if it stays in the window.
- For example, the agent invented `pandas.read_parquet_lazy()` a few turns back and is now calling it again.

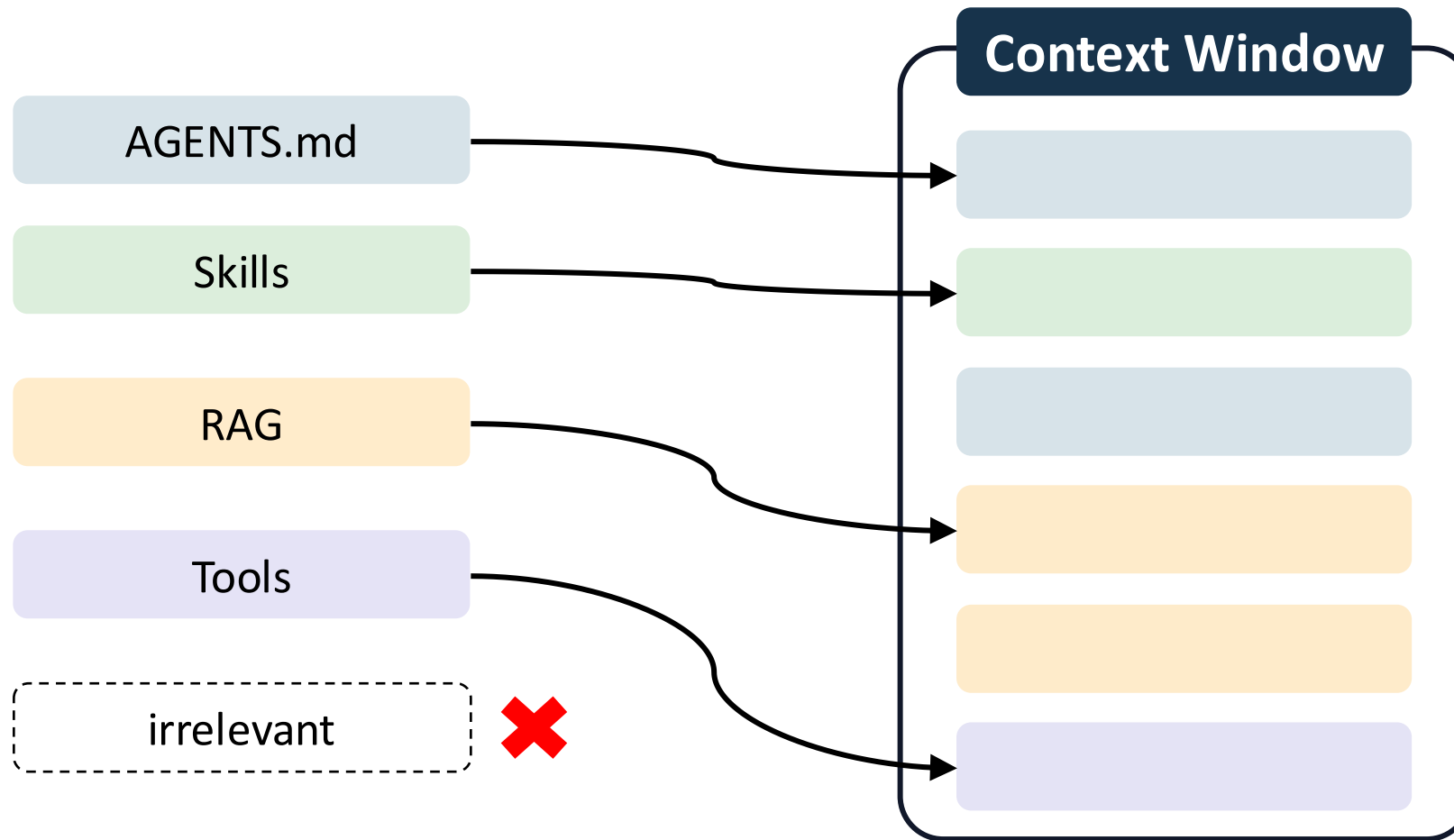
The agent keeps re-running the same failing command instead of stepping back to plan.

Most context-engineering techniques map to four strategies, each defined by a single question you can ask before any agent task.

- Curate
- Distill
- Delegate
- Externalize

Curate Strategy: Choose What Goes Into the Context

What should I put into the window for this turn?



■ Curate Strategy: Choose What Goes Into the Context

- The most common place to start is an instruction file.
- Auto-loaded files get pulled into every turn, so they're the right home for project-wide conventions.
- **A short AGENTS.md is plenty.** Long instruction files can become a context-bloat problem, and the agent tends to ignore any single rule when there are thirty of them.

AGENTS.md

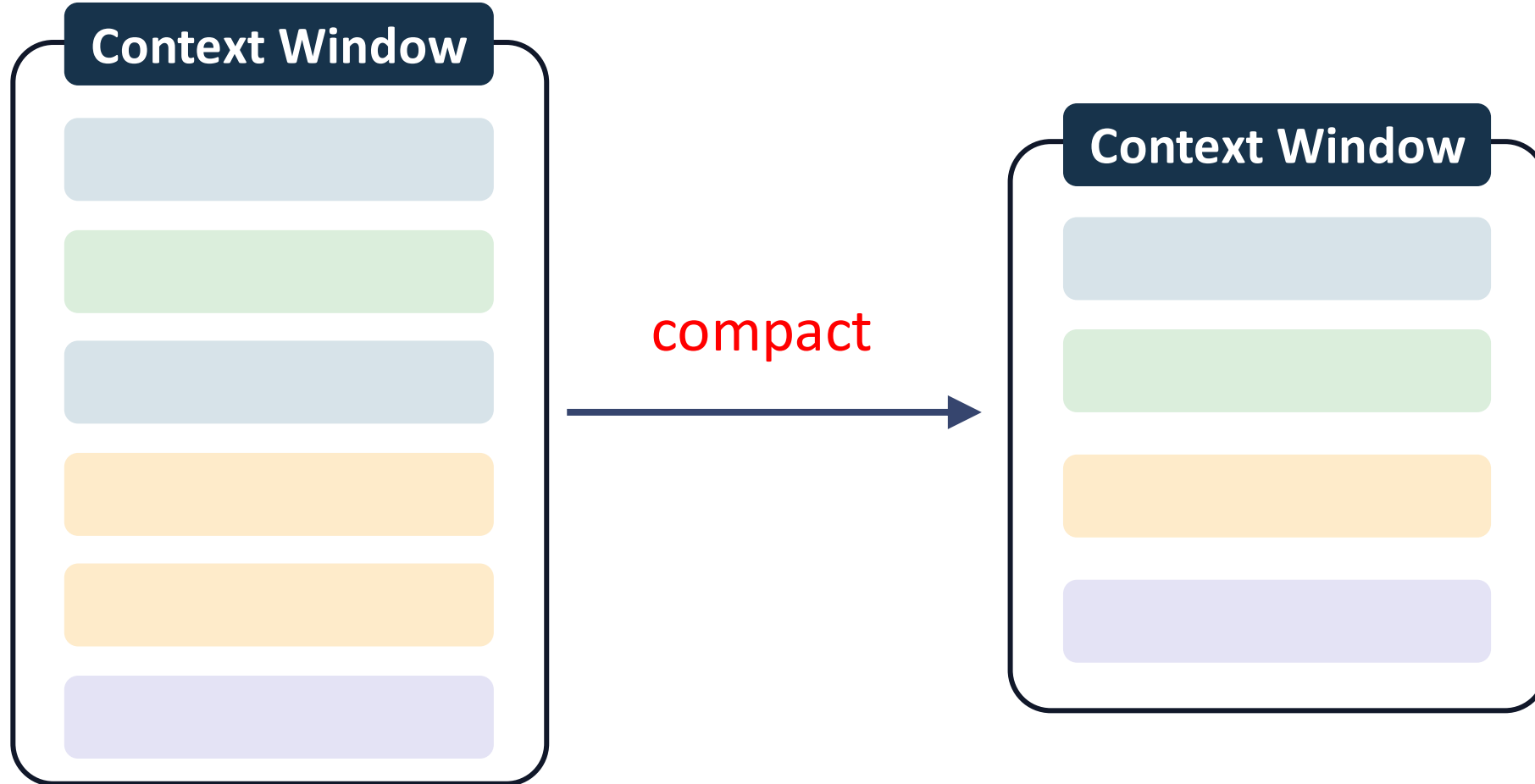
- Python version: 3.14. Don't assume newer syntax than that.
- Dependency manager: `uv`. Never use `pip`.
- Tests: `uv run pytest -q`. Add a test for every new function.
- Lint and format: `ruff check` and `ruff format`.
- Type checking: `mypy --strict`. Type-hint all public functions.
- Use Google-style docstrings.
- Don't add new dependencies without asking.

Curation tactics

- **Just-in-time file loading:** Let the agent search and open files as it needs them with tools like grep, glob, and file readers, instead of dumping the whole repo into the prompt up front. The agent's own searches pull in only the relevant slice.
- **Retrieval-augmented generation (RAG):** For docs, knowledge bases, and large codebases, you can use RAG to pull only the chunks that match the query instead of concatenating entire documents.
- **Skills:** A skill is a description of a workflow that the agent loads only when relevant for a given task. Skills keep the window light when most tasks don't need all the skills.

Distill Strategy: Shrink What's Already in the Window

What can I shrink without losing the point?



Distill tactics

- **Drop old or unneeded tool output:** Big tool results like full file dumps, long search results, and verbose tracebacks take up a lot of space and rarely need to be re-read. Ask the agent to summarize or drop earlier tool outputs before pruning the conversation itself.
- **Trim turns that no longer matter:** When you've moved on to a new feature or switched to a bug fix, older turns about the previous work mostly add noise. Trim them to free up room for more relevant information.

Distill tactics

- **Summarize on demand:** Agents do this automatically when the window fills, but with today's long context windows, quality degrades well before that point. Trigger compaction yourself at natural break points like finishing a sub-task or switching files.
- **Rewind to an earlier turn:** Some agents let you jump back and continue from a previous point in the conversation. That's useful when the recent few turns went sideways and you want to drop them without restarting the whole conversation.

Delegate Strategy: Split Work Across Separate Context Windows

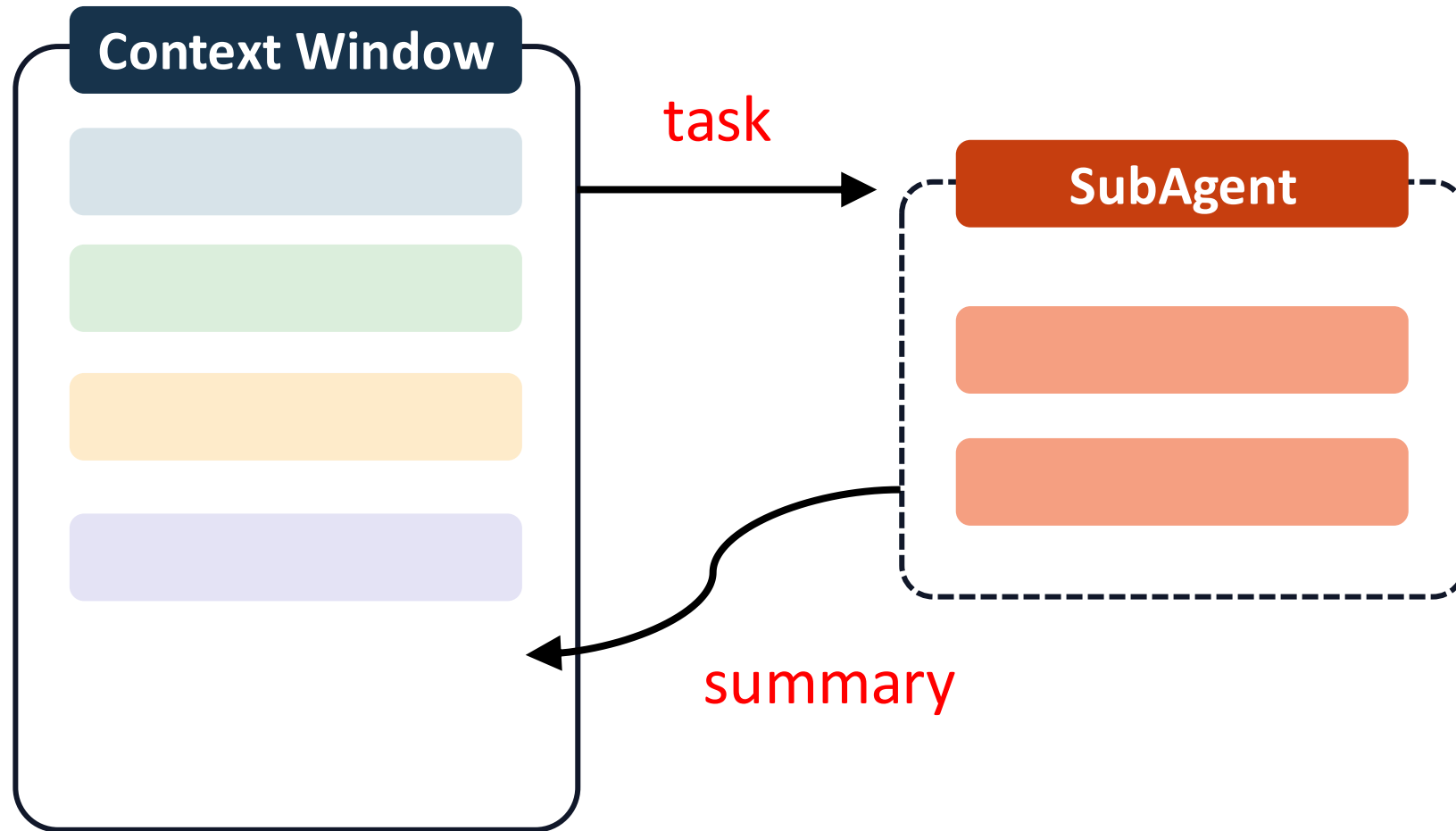
Sometimes the right approach is to **use more than one context window** instead of trying to fit everything into just one.

Delegate strategy consists of assigning a focused task to **a subagent with its own clean context window**.

- The subagent works through the task, loads whatever context it needs, and returns a short summary to the main agent.
- The main window stays lean, and the subagent's exploration never has to compete for attention with your top-level conversation.

■ Delegate Strategy: Split Work Across Separate Context Windows

What deserves its own separate window?

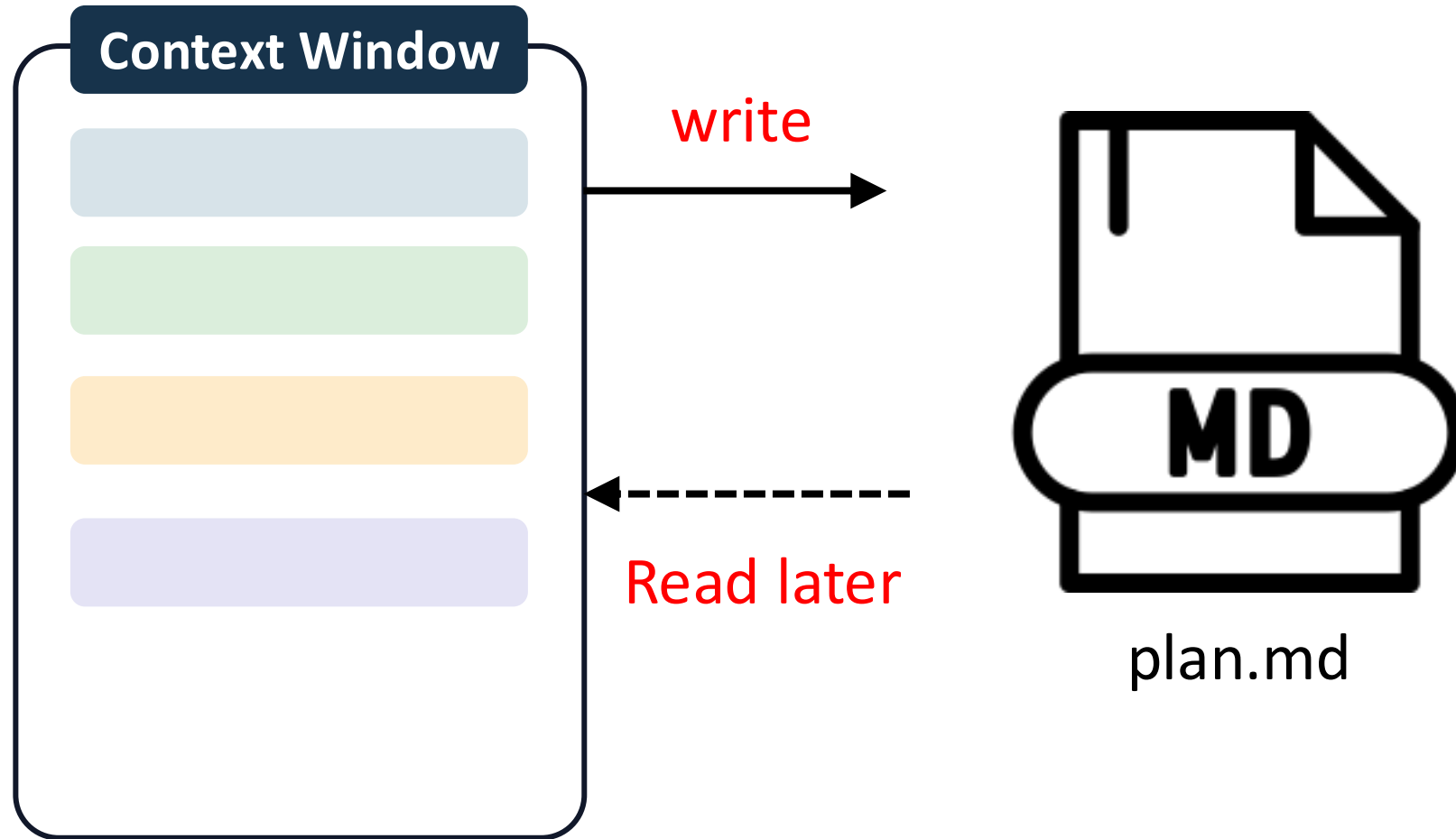


When to delegate to subagents:

- **Subagents for focused tasks:** A main agent delegates a job like “Write the test file for users.py” to a subagent. The subagent reads the relevant files, does the work, and returns a concise result. The main agent doesn’t have to see the subagent’s intermediate work, just the final answer.
- **Wrappers around noisy tools:** When a tool returns more data than the main agent needs, hand the call to a subagent. The subagent processes the response and returns only the relevant slice. This is useful for long file dumps, database queries, raw browser HTML, and similar.
- **Sandboxed code execution:** Run code in an isolated sandbox so only the result enters the agent’s window. Intermediate state, prints, and full tracebacks never reach the main context.

Externalize Strategy: Save Context the Agent Can Read Later

What should I save outside the window so the agent can pull it back later?



Externalize Strategy: Save Context the Agent Can Read Later

This strategy is especially useful for long-running work that requires multiple sessions, since the context you save to disk survives compactions and restarts.

Plan files: A plan.md file (or similar) on disk survives compaction and fresh sessions. The agent can re-read it on demand to refresh its memory about the task.

[One-shot Externalize prompt]

Write a short plan for this task into plan.md before you start, and then update plan.md after each step.

Externalize Strategy: Save Context the Agent Can Read Later

Notes and scratchpads: For longer-running work, treat the scratchpad like a structured note-taking system. Group sections by Decisions, Open Questions, and Next Steps. A fresh session can pick up where the last one left off by reading the notes back in.

```
[plan.md]
```

```
## Decisions
```

- Validate email syntax with a regex.
- Verify the domain via DNS.

```
## Open Questions
```

```
Should `validate_email()` live in `users.py` or move to `validators.py`?
```

```
## Next Steps
```

- Add tests for the email validator.
- Work the validator into the `User` class.

■ Externalize Strategy: Save Context the Agent Can Read Later

Persistent memory: Some agents auto-capture running facts about your project across sessions, like preferred libraries or tricky setup steps.

- Cursor calls this Memories, separate from its rules.
- Claude Code calls it auto memory and stores it in a MEMORY.md file, separate from CLAUDE.md.

Context-Management Strategies: Summary

Strategy	Question	Possible Actions
Curate	What should I put into the window for this turn?	AGENTS.md, file search, RAG, skill descriptions, ...
Distill	What can I shrink without losing the point?	summarize on demand, drop old tool output, trim stale turns
Delegate	What deserves its own separate window?	Subagents, sandboxed code execution
Externalize	What should I save outside the window so the agent can pull it back later?	Notes files, plan files, persistent memory

Workflow Type

- Core Agent Loop: read, reason, act, evaluate
- Coding Agent Workflows: IDE Agent, Terminal Agents, Pull Request Agents, Cloud Agents

Terminal Agent (Using GitHub Copilot CLI)

- Install & Authenticate With GitHub
- Slash Commands & Keyboard Shortcut
- Agent Modes (Standard & Plan & Autopilot), Sub-Agent
- Switch Between AI Providers

Pull Request Agents (Using GitHub Copilot code review)

- Request a Code Review (--reviewer @copilot), Review and Apply Copilot's Suggestions
- Configure Copilot for Project (repository settings & custom instructions)

Context Engineering for Coding Agent

- Context-Management Strategies: Curate, Distill, Delegate, Externalize



Course Wrap-up

Course Agenda

Week	Date	Title	
1	2026-06-15	Introduction	
1	2026-06-17	Advanced Python Fundamentals-I	Part 1: Advanced Python Fundamentals
2	2026-06-22	Advanced Python Fundamentals-II	
2	2026-06-24	Advanced Python Fundamentals-III	
3	2026-06-29	Machine Learning I – libraries, workflows, supervised v.s. unsupervised	Part 2: Python for ML/DL/LLM
3	2026-07-01	Machine Learning II – scikit-learn, evaluation, hyperparameter, ...	
4	2026-07-06	Deep Learning I: PyTorch (tensors, autograd, dataset & dataloader)	
4	2026-07-08	Deep Learning-II: architectures, CNN, RNN, transformers	
5	2026-07-13	Deep Learning-III: LLM, training scheme	
5	2026-07-15	AI-powered Application I: prompting, structured outputs, RAG, AI Agent	Part 3: AI-powered and Agentic App
6	2026-07-20	AI-powered Applications II: AI Coding Agent, course wrap-up	
6	2026-07-22	Final Project Presentation	Part 4: Assessment
7	2026-07-27	Final Exam	

- **Basic Concepts**
- **Basic Data Types:** The fundamental building blocks (integers, floats, booleans).
- **Operators:** The “verbs” that allow us to manipulate and compare data.
- **Collections:** Essential for organizing and storing groups of data (lists, tuples, strings, sets, dictionaries).
- **Type Conversion:** Crucial for adapting data to different needs and leveraging various data type features.
- **Dynamic Typing:** determine variable types at runtime, allowing flexible programming but incurring extra overhead and requiring attention to type-related errors

Control Flow

- Conditional Statements: if, elif, else, condition expression, conditional assignment
- Loop Statements: while, for, range(), enumerate(), list comprehension
- Control Statements: break, continue, else, pass

Exceptions

- Exception Hierarchy
- Exception Handling: try, except, else, finally, raise, assert

Functions

- Fundamentals: define/call, variable scope, return statement, docstring, type hint
- Parameters/Arguments: positional v.s. keyword, default, variable length (*args, **kwargs), Keyword-only and Positional-Only(* and /), parameter order
- Lambda Functions: use with higher-order functions (map, filter, sorted, reduce)

Classes and Objects

- Objected-Oriented Programming, Encapsulation
- Constructor/Destructor, Class/Object-level attributes
- Dunder Methods (`__lt__`, `__str__`, `__len__`, `__getitem__`)

Inheritance

- Subclass/Superclass, Polymorphism

Modules and Packages

- built-in modules, user-defined modules, import
- `__name__ == '__main__'`, packages

Advanced Iteration: Recursion & Generators

- yield statement

Libraries for AI Workflows

	Library	Core Idea	Primary Data Type	Typical Tasks
• Numpy	NumPy	Vectorized numerical computing	ndarray	Element-wise ops, broadcasting, LA, random
• SciPy	SciPy	Algorithms on arrays	ndarray/sparse	Optimize, stats tests, sparse ops, FFT
• Pandas	pandas	Tabular data wrangling	DataFrame/Series	I/O, cleaning, merge/join, groupby, features
	Matplotlib	Visualization backend	n/a	Line/scatter/bar, images, layout, export
• Matplotlib & Seaborn				

Machine Learning Basics

- Core Terminology
- Main Categories: Supervised/UnsupervisedSemi-supervised/Reinforcement Learning
- Core Workflow: Data, Model, Train&Eval, Predict
- Main Challenges: “bad algorithm” and “bad data”

Lecture 6: Python Programming for Machine Learning

ML Basics with Scikit-Learn

- Load Dataset, Split Data
- Preprocess Data: Scale, Normalization, Encoding, Discretization, Binarization, Imputation, Generating Polynomial Features, Dimensionality Reduction

Models in Scikit-Learn

- Supervised Learning Models: Linear/Logistic Regression, SVM, Decision Trees, MLPs
- Unsupervised Learning Models: K-Means, Agglomerative Clustering, ...

Model Evaluation & Selection

- Generalization: The Bias-Variance Trade-off
- Data-Splitting Protocol
- Evaluation Metric: MSE,, RMSE, MAE, log loss, ROC-AUC, PR-AP, Precision/Recall/F1
- Cross-Validation: Stratified K-Fold, Group K-Fold, Time Series Split, Nested Cross-Validation

Hyperparameter Optimization

- Search Algorithm: Grid Search, Random Search, Successive Halving
- Search Space Engineering

Lecture 7: Python Programming for Deep Learning

Deep Learning and PyTorch

- the need for deep learning frameworks

Tensors in PyTorch

- attributes, creating, reshaping, moving tensors to GPU
- tensor Operations: element-wise, broadcasting, in-place, indexing/slicing, concatenation, stack, split, aggregation, linear algebra

Autograd & Model Building

- computation graph, loss functions, backpropagation and autograd, optimizer
- building models with nn.Module
- model.train(), model.eval(), torch.no_grad()
- saving and loading models

Dataset & Dataloader

- build-in dataset, custom dataset

Deep Model Architecture

- Multi-Layer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Sequence Modeling and RNN
- Language Modeling and Transformer
- Library: Hugging Face

Training Strategy

- Regularization
- Normalization
- Weight Initialization & Optimization
- Library: WandB (Weights and Biases)

Advanced Prompting & Reasoning

- zero-shot, few-shot, delimiters, describe the request in numbered Steps
- Chain-of-Thought (CoT) Prompting, Output Formatting
- Role in Conversation
- Self-Consistency (SC)

AI Agent

- Agent Development Frameworks: LangChain
- External Memory: RAG
- Text Embeddings: SentenceTransformers
- Vector Database: ChromaDB
- Agent & Tool Use

Workflow Type

- Core Agent Loop: read, reason, act, evaluate
- Coding Agent Workflows: IDE Agent, Terminal Agents, Pull Request Agents, Cloud Agents

Terminal Agent (Using GitHub Copilot CLI)

- Install & Authenticate With GitHub
- Slash Commands & Keyboard Shortcut
- Agent Modes (Standard & Plan & Autopilot), Sub-Agent
- Switch Between AI Providers

Pull Request Agents (Using GitHub Copilot code review)

- Request a Code Review (--reviewer @copilot), Review and Apply Copilot's Suggestions
- Configure Copilot for Project (repository settings & custom instructions)

Context Engineering for Coding Agent

- Context-Management Strategies: Curate, Distill, Delegate, Externalize

Beyond This Course

This course is only the starting point.

- Continue writing code: real learning happens when you build real things.
 - Understand before you ask AI: AI is most useful when you know how to judge its answers.
 - Explore open-source projects: learning from good code is as important as writing your own.
 - Keep learning: new libraries, frameworks, and AI tools will continue to evolve.
-
- AI will make you faster.
 - Your understanding is what makes you valuable.

Next Lecture: Project Presentation

Presentation Order:

- Topic C → Topic B → Topic A
- Within each topic, groups will present in ascending group number order.

Each group has **up to 6 minutes** for the presentation.

- Please keep within the time limit; otherwise, the schedule may run behind.

1		Member 1	Member 2	Member 3	Topic
2	Group 1	Han LUO, 50026248	Yutong ZHANG, 50025311	Xinjie SHI, 50025189	Topic A: Text Classification Pipeline Forensics
3	Group 2	Yihe MA 50026183	Zhirui MIAO 50027785	Qidan ZHENG 50027125	Topic A: Text Classification Pipeline Forensics
4	Group 3	吴奕萱, 50026042	潘莉瑶, 50026092	(空)	Topic A: Text Classification Pipeline Forensics
5	Group 4	Junyu Pan 50027369	Yingshi Bai 50026494	Yu Cheng Chu 50024874	Topic A: Text Classification Pipeline Forensics
6	Group 5	Zhao TONG, 50025256	Xiaoyang LI, 50025660	Xu CHEN, 50027151	Topic A: Text Classification Pipeline Forensics
7	Group 6	Qixiang Hua, 50013071	Jiaxin Dong, 50012558	(空)	
8	Group 7	陶文祥 50025830	李清扬 50028038	赵婧涵 50027711	Topic B: Dataset Forensics and Label Audit
9	Group 8	YifeiZhang张逸飞 50027694	ZhixuanXu许智轩 50025804	ZitongTang唐梓桐 50025842	Topic A: Text Classification Pipeline Forensics
10	Group 9	蔡奕洋 50026949	刘易畅 50026872	荆璿熙 50027723	Topic C: SWE-bench Codebase Adoption
11	Group 10	JianghaoYANG,50028052	Heyuan CHEN,50026298	Haibin WANG,50027137	Topic A: Text Classification Pipeline Forensics
12	Group 11	王培栋 50025488	蔡骥博 50026717	蔡勋至 50027462	Topic A: Text Classification Pipeline Forensics
13	Group 12	张翊洋 50025517	王子腾 50025866	陈星烨 50027644	
14	Group 13	Vasco Pereira, 50050380	Xiaolong Qiao, 50025050	Huang Zizhou, 50026470	
15	Group 14	谭子木 50027618	欧卓希 50026652	(空)	Topic A: Text Classification Pipeline Forensics
16	Group 15	陈楚聪 50026418	朱恩正 50026315	李浩维 50026171	Topic B: Dataset Forensics and Label Audit
17	Group 16	杨颖娴, 50027319	徐卓尔, 50025098	钟艺丹, 50027072	Topic A: Text Classification Pipeline Forensics
18	Group 17	Mohamed Elshenawy , 5005	Fatma AbdelMaksoud , 5005064	Shrook Mostafa , 50050445	Topic A: Text Classification Pipeline Forensics
19	Group 18	赖家兴, 50025701	何家睿, 50013538	(空)	Topic A: Text Classification Pipeline Forensics
20	Group 19	Jialin CHEN	(空)	(空)	Topic C: SWE-bench Codebase Adoption
21	Group 20	冼泽恒, 50026212	梁柏榮, 50026274	(空)	Topic B: Dataset Forensics and Label Audit
22	Group 21	Yufan XIAO, 50013241	(空)	(空)	Topic A: Text Classification Pipeline Forensics
23	Group 22	50025048张志凌	(空)	(空)	Topic A: Text Classification Pipeline Forensics
24	Group 23	QING Chen, 50025971	(空)	(空)	Topic B: Dataset Forensics and Label Audit
25	Group 24	梁君豪, 50026157			Topic A: Text Classification Pipeline Forensics

Next Lecture: Project Presentation

Project presentation evaluates **the quality of the presentation**, not the technical correctness of the project, which is assessed separately through the code and report review.

Presentation Rubric (10 points)

- Organization & Clarity(2): Logical structure and clear explanation of the problem, method, and results.
- Technical Understanding(3): Demonstrates the understanding of the approach and key design.
- Results Communication(3): Clearly presents key results and a representative case.
- Reflection (1): Discusses difficulties, AI usage, verification, limitations, and takeaways.
- Delivery&Time Management (1): Clear delivery, readable slides, and finishes within the time limit.

The presentation accounts for **25% of the final project grade**.

Python Programming with Artificial Intelligence

Thank you for listening!

